



19-OBSERVABILITY-AND-FORENSICS.md — Every failure debuggable from one artefact



The goal: when a photographer says *"it went wrong on Saturday"*, one click produces a file that tells you exactly what happened — without a single client photograph leaving their machine.

1. The four signals

Signal	Lives	Retention	Contains PII?
Structured logs	Local rolling files	7 runs or 500 MB	Never — redacted at the sink
Metrics	Local SQLite <code>metrics</code> table	90 days	No
Decision ledger	Project catalog	Life of the project	Photo IDs only, no faces
Run bundle	Generated on demand or on failure	User-controlled	No — opt-in for thumbnails

Telemetry to the vendor is **opt-in, aggregate only, and schema-allow-listed**.
Local observability is always on — that is what makes support possible offline.

2. Structured logging rules

- JSON lines, one event per line, no free-text-only logs.
- Mandatory fields: `ts`, `level`, `code` (or `event`), `run_id`, `task_id`, `photo_id`, `stage`, `duration_ms`, `ep`, `model_version`.

- Correlation: `run_id` flows through every layer including worker processes and the UI. Any log line without a `run_id` during a run is a bug.
- Levels are meaningful: `error` = a coded failure, `warn` = degraded path taken, `info` = stage transitions only, `debug` = per-photo, `trace` = per-tile (never on by default).
- **Cardinality discipline** — no file paths, no filenames, no user text in metric labels. Paths go in the detail field of the log, which is redacted before export.

3. Redaction (non-negotiable)

A single redaction filter sits in front of every sink — logs, crash reports, telemetry, run bundles. It removes:

- API keys and anything matching common key patterns (belt and braces: the key should never reach a log anyway).
- Absolute paths, replaced with `<catalog>/...` or `<import>/...`.
- Filenames, replaced with the photo UUID.
- People names, project names, venue names — replaced with stable hashed IDs.
- EXIF free-text fields, which are treated as untrusted input and never echoed.

A CI test feeds a poisoned fixture containing a fake key, a real-looking path, and a guest name through every sink and asserts none of them appear in any output.

4. The decision ledger

This is the product feature and the debugging tool at once — the same table powers "Explain My Edit" and post-mortems.

```
-- FROZEN CONTRACT v1
CREATE TABLE decision_ledger (
  id          INTEGER PRIMARY KEY,
  run_id      TEXT NOT NULL,
  photo_id    INTEGER,
  stage       TEXT NOT NULL,      -- 'cull' | 'wb' | 'ret
```

```

ouch' | 'qc' ...
  decision      TEXT NOT NULL,      -- 'keep' | 'reject' |
'applied' | 'escalated'
  reason_code   TEXT NOT NULL,      -- enumerated, translat
able, never free text
  evidence      TEXT NOT NULL,      -- JSON: the numbers th
at drove it
  confidence    REAL NOT NULL,
  autonomy_band TEXT NOT NULL,      -- 'auto' | 'zero_touc
h' | 'suggest' | 'review'
  model_set     TEXT NOT NULL,
  source        TEXT NOT NULL,      -- 'local' | 'cloud' |
'cloud_cache' | 'fallback'
  reverted_at   INTEGER,            -- set when the human o
verrides
  created_at    INTEGER NOT NULL
);

```

Every automated decision writes exactly one row. Reversibility is checked by a test: for each `decision`, applying the inverse must restore the prior state exactly.

5. Metrics that matter

Metric	Type	Why
<code>stage.duration_ms</code>	Histogram by stage, tier	Budget enforcement
<code>task.retries</code>	Counter by code	Early warning of a flaky environment
<code>item.quarantined</code>	Counter by code	Systemic failure detection (F-02, F-03)
<code>ep.active</code>	Gauge	Catches silent CPU fallback (F-18)
<code>gpu.device_lost</code>	Counter	Driver health
<code>memory.rss_peak</code>	Gauge per run	Ceiling enforcement
<code>cloud.calls</code> / <code>cloud.usd</code>	Counter	Spend governance

Metric	Type	Why
<code>decision.override_rate</code>	Ratio by stage, model version	The single best drift signal
<code>run.completed</code> / <code>run.aborted</code>	Counter by reason	Reliability headline
<code>ui.frame_time</code>	Histogram	Interaction budgets

6. The run bundle

One zip, generated by **Help** → **Create diagnostic bundle** or automatically on

Fatal :

```

run-2026-08-09-a1b2c3.zip
  manifest.json          app version, OS, CPU/GPU, driver, EP, model set hashes
  run-summary.json       counts, durations, budgets measured vs ceiling
  logs/*.jsonl           redacted structured logs for this run
  metrics.json           all metrics for this run
  ledger.csv             decision ledger (IDs only)
  errors.json            every AuraError with code, context, attempt
  graph.json             final task-graph state – the most useful file in the bundle
  budgets.json           measured vs target vs ceiling
  env.txt                relevant env vars, redacted
  thumbnails/            EMPTY unless the user ticks "include 12 thumbnails"

```

Rules: bundle generation must never fail (best-effort per section, with a note if a section is missing), must complete in under 10 s, and must be under 20 MB without thumbnails. It is written to the user's chosen folder and never auto-uploaded.

7. Crash handling

- Out-of-process reporter, so it survives the crash (F-38).
- On crash: write a breadcrumb file first, then a minidump, then attempt the run bundle.
- On next launch: *"AURA closed unexpectedly during Sunset portraits. Your work is intact — resume from 68 %?"* with a **Show what happened** link. Never a raw stack trace, never a lost run.
- Symbols are uploaded to the symbol server at release time so any minidump is symbolizable later.

8. Support workflow

1. Photographer hits a problem, clicks **Create diagnostic bundle**.
2. Support reads `run-summary.json` and `graph.json` first — the failing task and its dependency chain are visible in seconds.
3. `errors.json` gives the code, which maps to a runbook in `docs/runbooks/<code>.md`.
4. If reproduction is needed, `manifest.json` pins the exact app, driver, EP and model versions.
5. Any new failure mode gets an entry in `14-FMEA-FAILURE-CATALOG.md` and a regression test. **A bug that has no test has not been fixed.**

9. Acceptance criteria

- ☐ Every log line during a run carries `run_id`; test asserts zero orphan lines.
- ☐ Poisoned-fixture redaction test passes across all sinks.
- ☐ Every decision writes exactly one ledger row; reversibility test green.
- ☐ Run bundle generates in under 10 s, under 20 MB, and never throws.
- ☐ Crash reporter proven by a deliberate crash test on all three platforms.
- ☐ Every shipped error code has a runbook file.

10. Claude Code prompt

Act as `DEVOPS`. Implement structured logging with the mandatory field set, the single redaction filter in front of every sink, the metrics table, and the run bundle generator exactly as described in `19-OBSERVABILITY-AND-FORENSICS.md`. Implement the `decision_ledger` table verbatim as the frozen SQL. Then switch hats to `SEC` and write the poisoned-fixture redaction test covering logs, crash reports, telemetry and the run bundle. Report PASS/FAIL with command output. The bundle generator must never panic — prove it by injecting a failure into each section.